

# SENGUICHET

## Couche Données

*Architecture, Tables, Requêtes & Propositions*

Version 1.0 • Mai 2026

## 1. Vue d'ensemble de la couche données

La couche données de SENGUICHET repose sur **4 systèmes de stockage** complémentaires, chacun ayant un rôle précis dans l'architecture n-tiers. Aucun système ne peut être remplacé par un autre — ils coexistent et se synchronisent.

| Système           | Technologie           | Rôle principal                                                                                    |
|-------------------|-----------------------|---------------------------------------------------------------------------------------------------|
| MySQL             | Base relationnelle    | Stockage permanent de toutes les entités métier : utilisateurs, événements, billets, transactions |
| SQLite            | Base embarquée mobile | Copie temporaire des billets pour le mode hors-ligne du contrôleur                                |
| Redis             | Cache en mémoire      | Stockage éphémère : codes OTP, sessions JWT, verrous anti-doublon scan                            |
| Stockage fichiers | S3 / Cloudinary       | Affiches d'événements, QR codes générés — MySQL stocke seulement l'URL                            |

## 2. MySQL — Base de données principale

MySQL centralise toutes les données permanentes de SENGUICHET. Voici les tables issues du MLD, regroupées par domaine fonctionnel, avec les propositions d'amélioration identifiées.

### 2.1 Domaine Utilisateurs

| Table          | Colonnes clés                               | Statut MLD                                   | Notes                                          |
|----------------|---------------------------------------------|----------------------------------------------|------------------------------------------------|
| acheteur       | id, numeroTel, statut                       | Conforme <input checked="" type="checkbox"/> | Identifié par numéro de téléphone uniquement   |
| organisateur   | id, nom, numTel, email, motsDePasse, status | Conforme <input checked="" type="checkbox"/> | Compte complet avec authentification email/mdp |
| controleur     | id, #idEvenement, codeAcces, status         | Conforme <input checked="" type="checkbox"/> | Lié à un événement via code d'accès temporaire |
| administrateur | id, numTel, email, motsDePasse              | Conforme <input checked="" type="checkbox"/> | Super-utilisateur de la plateforme             |

#### Proposition d'amélioration

Proposition : Ajouter une table 'session\_utilisateur' (id, userId, token\_jwt, date\_expiration, ip\_adresse) pour tracer les connexions actives et permettre la déconnexion forcée par l'administrateur. Non présente dans le MLD mais essentielle pour la sécurité.

### 2.2 Domaine Événements

| Table            | Colonnes clés                                                                                                  | Statut MLD                                   | Notes                                                           |
|------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------------------------|
| evenement        | id, #idOrganisateur, titre, description, lieu, Date_evenement, CapaciteTotale, PlaceRestantes, status, affiche | Conforme <input checked="" type="checkbox"/> | La colonne 'affiche' stocke l'URL du fichier sur S3             |
| categorie_ticket | id, #id_evenement, nom, prix, QuantiteDisponible, QuantiteVendue                                               | Conforme <input checked="" type="checkbox"/> | Permet plusieurs catégories par événement (VIP, Standard, etc.) |

#### Proposition d'amélioration

Proposition : Ajouter une table 'media\_evenement' (id, #idEvenement, url, type ENUM[AFFICHE, BANNIERE, GALERIE], ordre, taille\_ko, date\_upload) plutôt que stocker l'affiche directement dans 'evenement'. Cela permettra d'avoir plusieurs photos par événement en V2 sans modifier la table principale.

## 2.3 Domaine Billetterie

| Table         | Colonnes clés                                                                                                                | Statut MLD                                   | Notes                                                             |
|---------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-------------------------------------------------------------------|
| billet        | id, #id_acheteur, #idCategorie, Reference, qrCode, qrCodeSecret, status, dateScan, dateAchat, dateExpiration, #idTransaction | Conforme <input checked="" type="checkbox"/> | qrCodeSecret est la clé anti-fraude — jamais exposée à l'acheteur |
| transaction   | id, #id_acheteur, montant, Reference, operateur, numeroPayeur, status, dateTransaction                                       | Conforme <input checked="" type="checkbox"/> | Référence unique pour réconciliation avec les APIs Wave/Orange    |
| remboursement | id, #id_Acheteur, code, montant, numDestinataire, statut, tentative, dateRemboursement, #idBillet                            | Conforme <input checked="" type="checkbox"/> | Compteur de tentatives pour les relances automatiques             |

### Proposition d'amélioration

Proposition : Ajouter une colonne 'lien\_annulation' et 'lien\_expiration' dans la table 'billet'. Le lien d'annulation est un token unique envoyé par SMS qui permet à l'acheteur d'annuler sans créer de compte. Ce mécanisme est absent du MLD actuel mais critique pour la gestion des remboursements sans compte.

## 2.4 Domaine Sécurité & Notifications

| Table        | Colonnes clés                                                                      | Statut MLD                                   | Notes                                                         |
|--------------|------------------------------------------------------------------------------------|----------------------------------------------|---------------------------------------------------------------|
| codeOTP      | id, #idAcheteur, code, dateExpiration, EstUtilise                                  | Conforme <input checked="" type="checkbox"/> | Si Firebase Auth est utilisé, cette table devient optionnelle |
| notification | id, #id_acheteur, #id_Organisateur, Type, destinataire, contenu, status, dateEnvoi | Conforme <input checked="" type="checkbox"/> | Logger chaque SMS envoyé via AfricasTalking pour le suivi     |

### Proposition d'amélioration

Proposition : Ajouter une table 'log\_activite' (id, action, acteur\_type, acteur\_id, entite, details, date\_action, ip\_adresse) absente du MLD. Indispensable pour l'audit de sécurité : tracer les validations d'organismes, les remboursements forcés, les suspensions de comptes par l'admin.

## 3. SQLite — Base locale du contrôleur

SQLite est une base de données légère stockée directement sur le téléphone du contrôleur. Elle ne contient que les données strictement nécessaires pour scanner les billets hors-ligne. Elle est vidée après chaque événement.

### 3.1 Tables SQLite

| Table         | Colonnes                                                                                           | Rôle                                                                         |
|---------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| billets_cache | qr_code TEXT (PK), statut TEXT DEFAULT 'VALIDE', nom_evenement TEXT, heure_limite TEXT             | Copie locale des billets valides pour vérification sans connexion            |
| sync_meta     | id INTEGER (PK), evenement_id TEXT, derniere_sync DATETIME, nb_billets INTEGER, nb_scannes INTEGER | Métadonnées de synchronisation pour affichage dans l'interface du contrôleur |

#### Proposition d'amélioration

Proposition : Ajouter une colonne 'scan\_timestamp' dans 'billets\_cache' pour enregistrer l'heure exacte du scan hors-ligne. Lors de la synchronisation avec MySQL, cette heure sera utilisée pour résoudre les conflits (si deux contrôleurs ont scanné le même billet offline).

### 3.2 Cycle de vie de SQLite

| Étape                       | Action                                                       | Déclencheur                                              |
|-----------------------------|--------------------------------------------------------------|----------------------------------------------------------|
| 1. Initialisation           | Création de la table billets_cache si inexistante            | Première ouverture de l'app contrôleur                   |
| 2. Synchronisation entrante | Chargement de tous les QR codes valides depuis MySQL via API | Contrôleur entre le code événement avec connexion active |
| 3. Scan hors-ligne          | Lecture et mise à jour de statut dans billets_cache          | Chaque scan de QR code, avec ou sans connexion           |
| 4. Synchronisation sortante | Envoi des statuts SCANNE vers MySQL                          | Automatique dès reconnexion réseau (NetInfo)             |
| 5. Nettoyage                | Suppression de tous les enregistrements de billets_cache     | Fin de l'événement ou déconnexion du contrôleur          |

## 4. Redis — Cache en mémoire

Redis stocke des données temporaires avec expiration automatique. Rien dans Redis n'est permanent — si Redis redémarre, tout est perdu sans impacter l'intégrité des données (tout est aussi dans MySQL).

### 4.1 Structure des clés Redis

| Clé (pattern)                 | Valeur                | TTL                     | Rôle                                                         |
|-------------------------------|-----------------------|-------------------------|--------------------------------------------------------------|
| otp:{numeroTel}               | code à 6 chiffres     | 5 minutes               | Code OTP en attente de vérification                          |
| otp_attempts:{numeroTel}      | compteur entier (0-3) | 15 minutes              | Nombre de tentatives échouées — blocage à 3                  |
| session:{userId}              | token JWT (string)    | 24 heures               | Session active d'un organisateur ou admin                    |
| scan_lock:{qrCode}            | 1 (flag)              | 2 secondes              | Verrou anti-doublon lors du scan simultané par 2 contrôleurs |
| event_billets_count:{eventId} | entier                | 1 heure                 | Cache du nombre de billets vendus pour le dashboard          |
| blacklist_token:{jti}         | 1 (flag)              | durée restante du token | Token JWT révoqué (déconnexion explicite)                    |

#### Proposition d'amélioration

Proposition : Ajouter la clé 'rate\_limit:payment:{numeroTel}' avec TTL de 1 minute et compteur max à 3. Cela empêche un utilisateur de spammer les tentatives de paiement, ce qui pourrait générer des frais API excessifs côté Wave/Orange Money.

## 5. Stockage fichiers — S3 / Cloudinary

Les fichiers binaires (images, QR codes) ne sont jamais stockés dans MySQL. Ils sont hébergés sur un service cloud, et MySQL ne stocke que l'URL d'accès.

### 5.1 Organisation des dossiers

| Chemin                               | Contenu                             | Généré par                                    |
|--------------------------------------|-------------------------------------|-----------------------------------------------|
| /affiches/evenement_{id}.jpg         | Affiche uploadée par l'organisateur | Upload manuel lors de la création d'événement |
| /qrcores/billet_{reference}.png      | QR code Smart Ticket de l'acheteur  | Auto-généré après confirmation du paiement    |
| /thumbnails/evenement_{id}_thumb.jpg | Miniature pour liste des événements | Généré automatiquement par Cloudinary         |

#### Proposition d'amélioration

Proposition : Utiliser Cloudinary plutôt qu'AWS S3 pour la V1 — plan gratuit jusqu'à 25 Go, génération automatique de miniatures via URL transformée, pas besoin de configurer des buckets. Pour passer en production à grande échelle, migrer vers S3 sera trivial car seules les URLs dans MySQL changent.

## 6. Catalogue des requêtes inter-systèmes

Voici l'inventaire des requêtes échangées entre les différents systèmes de stockage, organisées par flux fonctionnel.

### 6.1 MySQL ↔ Redis

| Requête                                | Direction       | Flux fonctionnel                             | Fréquence                    |
|----------------------------------------|-----------------|----------------------------------------------|------------------------------|
| SET otp:{tel} code EX 300              | Backend → Redis | Génération code OTP après saisie du numéro   | Chaque connexion acheteur    |
| GET otp:{tel}                          | Backend ← Redis | Vérification du code saisi par l'acheteur    | Chaque tentative OTP         |
| DEL otp:{tel}                          | Backend → Redis | Invalidation après utilisation ou expiration | Après vérification réussie   |
| INCR otp_attempts:{tel}                | Backend → Redis | Comptage des tentatives échouées OTP         | Chaque OTP incorrect         |
| SETEX session:{userId} 86400 token     | Backend → Redis | Création session JWT organisateur/admin      | Chaque connexion réussie     |
| GET session:{userId}                   | Backend ← Redis | Vérification session sur chaque requête API  | Chaque appel API authentifié |
| DEL session:{userId}                   | Backend → Redis | Déconnexion explicite                        | Déconnexion utilisateur      |
| SET scan_lock:{qrCode} 1 EX 2          | Backend → Redis | Verrou anti-doublon avant scan               | Chaque scan de billet        |
| EXISTS scan_lock:{qrCode}              | Backend ← Redis | Vérification si scan déjà en cours           | Chaque scan de billet        |
| SET event_billets_count:{id} n EX 3600 | Backend → Redis | Mise en cache du compteur de ventes          | Après chaque achat           |
| GET event_billets_count:{id}           | Backend ← Redis | Lecture du compteur pour dashboard           | Chaque affichage dashboard   |
| SADD blacklist_token:{iti} 1           | Backend → Redis | Révocation token lors déconnexion forcée     | Déconnexion admin/suspend    |

### 6.2 MySQL ↔ Stockage fichiers

| Requête                            | Direction               | Flux fonctionnel                             | Table MySQL impactée |
|------------------------------------|-------------------------|----------------------------------------------|----------------------|
| UPLOAD affiche multipart/form-data | Backend → S3/Cloudinary | Organisateur upload l'affiche de l'événement | evenement.affiche    |



|                               |                         |                                                   |                   |
|-------------------------------|-------------------------|---------------------------------------------------|-------------------|
| GET url_signée affiche        | Backend ← S3/Cloudinary | Récupération URL après upload réussi              | evenement.affiche |
| DELETE affiche/{evenement_id} | Backend → S3/Cloudinary | Suppression lors d'annulation d'événement         | —                 |
| UPLOAD qrcode PNG généré      | Backend → S3/Cloudinary | Enregistrement QR code après paiement confirmé    | billet.qrCode     |
| GET url qrcode/{reference}    | Backend ← S3/Cloudinary | URL du billet pour envoi SMS à l'acheteur         | billet.qrCode     |
| GET thumbnail/{evenement_id}  | App mobile ← CDN        | Chargement miniature dans la liste des événements | —                 |

### 6.3 MySQL ↔ SQLite (via backend + app contrôleur)

| Requête                                                           | Direction        | Flux fonctionnel                            | Déclencheur                      |
|-------------------------------------------------------------------|------------------|---------------------------------------------|----------------------------------|
| SELECT id, qr_code FROM billet WHERE #idEvenement = ?             | MySQL → Backend  | Récupération billets avant événement        | Entrée code événement contrôleur |
| BULK INSERT INTO billets_cache VALUES (...)                       | Backend → SQLite | Chargement en masse dans la base locale     | Après SELECT MySQL réussi        |
| SELECT * FROM billets_cache WHERE statut = 'VALIDE'               | App → SQLite     | Vérification validité billet au scan        | Chaque scan QR code              |
| UPDATE billets_cache SET statut='SCANNE' WHERE qr_code=?          | App → SQLite     | Marquage billet comme utilisé en local      | Scan valide confirmé             |
| SELECT * FROM billets_cache WHERE statut='SCANNE'                 | App → SQLite     | Récupération scans à synchroniser           | Reconnexion réseau détectée      |
| UPDATE billet SET status='UTILISE', dateScan=NOW() WHERE qrCode=? | Backend → MySQL  | Synchronisation scans hors-ligne vers MySQL | Après reconnexion                |
| DELETE FROM billets_cache                                         | App → SQLite     | Nettoyage après synchronisation complète    | Fin d'événement                  |
| SELECT COUNT(*) FROM billets_cache WHERE statut='SCANNE'          | App → SQLite     | Affichage compteur scans dans l'interface   | Rafraîchissement UI              |

### 6.4 MySQL ↔ APIs externes (Wave / Orange Money / AfricasTalking)

| Requête                                   | Direction                | Flux fonctionnel                          | Table MySQL impactée                |
|-------------------------------------------|--------------------------|-------------------------------------------|-------------------------------------|
| POST /v1/checkout (Wave API)              | Backend → Wave           | Initiation paiement acheteur              | transaction.status = PENDING        |
| POST webhook paiement confirmé            | Wave → Backend           | Confirmation réception paiement           | transaction.status = CONFIRMED      |
| POST webhook paiement échoué              | Wave → Backend           | Notification échec paiement               | transaction.status = FAILED         |
| POST /v1/transfer remboursement (Wave)    | Backend → Wave           | Déclenchement remboursement vers acheteur | remboursement.statut = PENDING      |
| POST webhook remboursement confirmé       | Wave → Backend           | Confirmation remboursement effectué       | remboursement.statut = SUCCESS      |
| POST /1/messages (AfricasTalking SMS)     | Backend → AfricasTalking | Envoi SMS billet ou code OTP              | notification.status                 |
| <a href="#">GET callback delivery SMS</a> | AfricasTalking → Backend | Confirmation livraison ou échec SMS       | notification.status = ENVOYE/ECHOUÉ |

## 7. Synthèse des recommandations

Voici les 5 tables ou colonnes proposées en supplément du MLD initial, avec leur priorité d'implémentation pour la V1.

| Proposition                 | Type                   | Priorité V1 | Justification                                                                          |
|-----------------------------|------------------------|-------------|----------------------------------------------------------------------------------------|
| session_utilisateur         | Nouvelle table MySQL   | Haute       | Sécurité obligatoire : gestion des JWT et déconnexion forcée par admin                 |
| lien_annulation dans billet | Nouvelle colonne MySQL | Haute       | Sans cette colonne, les acheteurs sans compte ne peuvent pas demander de remboursement |
| media_evenement             | Nouvelle table MySQL   | Moyenne     | Prépare la V2 multi-photos sans breaking change sur la table evenement                 |
| log_activite                | Nouvelle table MySQL   | Moyenne     | Audit de sécurité indispensable pour le rapport de stage et la production              |
| rate_limit:payment Redis    | Nouvelle clé Redis     | Haute       | Protège contre les abus API Wave/Orange qui coûtent de l'argent réel                   |
| sync_meta SQLite            | Nouvelle table SQLite  | Moyenne     | Améliore l'UX du contrôleur : affiche les stats de scan en temps réel                  |

*Ce document constitue la spécification complète de la couche données SENGUICHET. Il doit être utilisé comme référence lors de l'implémentation des repositories en Semaine 2 et des migrations de base de données en Semaine 3.*

SENGUICHET • Version 1.0 • Mai 2026